

Chapter 6 One-Pager — End-to-End Game AI Architectures

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

June 2026

Problem. Chapters 2-5 assembled parts in isolation — CFR variants, abstraction, neural value approximation — without asking how they compose into a system that beats humans, or what such a system guarantees. Chapter 6 surveys the five competition-grade architectures of 2017-2023 as an evolution of design trade-offs rather than a leaderboard: the state of the art this dissertation builds on and departs from.

Approach. A deliberately *architectural* study — what each system computes offline, what it computes in real time, where learning sits, what it can prove — of **DeepStack** (2017), **Libratus** (2017/18), **Pluribus** (2019), **ReBeL** (2020) and **Student of Games** (2023), each scored on the same nine dimensions and placed on three axes: abstraction to neural representation, offline precomputation to real-time search, imperfect-information-only to unified play. **This chapter produced no code; every number below is the result each paper reports, not a measurement of ours.**

Key results (as reported by the papers).

- *The abstraction era was far more exploitable than its match results suggested.* A local-best-response probe showed top competition bots losing **over 3,000 mbb/g** — four times what folding every hand would lose. Closing that gap is what the decade was for.
- *Search at inference, not the offline strategy, carries the edge.* Libratus’s raw blueprint **did not beat** Baby Tartanian8 (-8 ± 15 mbb/g); with nested safe subgame solving on, the same system **won by 63**, and beat four professionals by **147 mbb/g over 120,000 hands**. Solving off-menu bets live also beat nearest-size action translation **119 vs 1,465 mbb/g** worst-case — Chapter 4’s translation failure repaired by refusing to translate.
- *Cost is not the axis it looks like.* DeepStack spent **~175 CPU-core-years** labelling values and Libratus **~25M core-hours**, while Pluribus’s blueprint cost **~12,400 core-hours** — about **\$150** on one server — and still beat five elite professionals at **+48 mbb/g**.
- *Every advance was paid for elsewhere.* Pluribus reached six players by discarding **all** safety guarantees for unsafe search; ReBeL recovered the two-player guarantee and eliminated abstraction and blueprint alike (Dong Kim **+165 mbb/g**) but retreated to two players; Student of Games unified perfect- and imperfect-information play soundly (beating the LBR probe by **+434 mbb/g**) while winning only **2 of 400** Go games against AlphaZero — its authors’ “price of generality”. No system dominates every axis.
- *The one omission all five share.* Each is **opponent-blind by design**: it computes a worst-case-robust strategy and plays it unconditionally. Libratus and Pluribus say so explicitly, and Pluribus does not even know who it is playing.

Thesis connection. That shared omission is the opening this dissertation occupies, and the chapter supplies the machinery to fill it. ReBeL’s **public belief state** is the substrate Contribution #1 widens from beliefs over cards to beliefs over opponent *type*, inheriting the soundness proofs attached to it rather than bolting adaptation on outside. Pluribus is Contribution #2’s argument in one system: if equilibrium buys no safety at $N > 2$ anyway, declining to exploit forfeits a guarantee never held. Exploitability, the LBR probe (repurposed from certificate to stress test) and AIVAT variance reduction are three ready instruments for Contribution #3.

Open questions. Four, straight from the synthesis: opponent-blindness itself (Chapter 7); safe exploitation beyond two-player zero-sum, which Pluribus won without and Student of Games could not extend (Chapter 8); carrying an opponent model *past the depth limit* instead of discarding it at the leaf; and real-time compute budgets, whose six-order-of-magnitude spread binds hardest on an agent that must re-solve *and* re-estimate an opponent at once. Corollary: build on the cheap, open lineage — the flagship code is unreleased or supercomputer-scale.